

Содержание

ВВЕДЕНИЕ	2
1 ОРГАНИЗАЦИОННАЯ СТРУКТУРА УПРАВЛЕНИЯ И БИЗНЕС ПРЕД- ПРИЯТИЯ.....	3
1.1 Общие сведения о предприятии. Организационная структура управле- ния.....	3
1.2 Анализ бизнес-процессов.....	5
1.3 Анализ парка вычислительной техники и программное обеспечение.....	6
2 НАСТРОЙКА НЕОБХОДИМОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И РАЗРАБОТКА КОДА	9
2.1 Установка среды программирования и компилятора.....	9
2.2 Реализация защищенного мессенджера на С++ и С#.....	11
3 СРАВНИТЕЛЬНОЕ ИССЛЕДОВАНИЕ ПРОИЗВОДИТЕЛЬНОСТИ.....	12
Заключение.....	14
Список литературы.....	15
Приложения	16

ВВЕДЕНИЕ

Настоящая практика включена в утвержденный рабочий учебный план обучающихся Аккредитованного образовательного частного учреждения высшего образования «Московский финансово-юридический университет МФЮА» по направлению подготовки 09.03.03 «Прикладная информатика», профиль «Прикладная информатика в экономике».

Вид практики: Учебная практика.

Тип практики: Учебная научно-исследовательская практика.

В качестве места прохождения практики приказом ректора закреплено: Аккредитованное образовательное частное учреждение высшего образования Ярославский филиал «Московский финансово-юридический университет МФЮА»;

расположенное по адресу: Ярославль, Кузнецова, д 1;

а также указан нормативный срок прохождения данной практики в период с «1» июня 2026 г. по «29» июня 2026 г.

Цель ознакомительной практики:

-научиться реализовывать программу на C++ для работы с локальной сетью.

Задачи ознакомительной практики:

Задачами ознакомительной практики по направлению подготовки 09. 03. 03 «Прикладная информатика», профиль «Прикладная информатика в экономике» являются:

- Сравнить возможности работы с сетью в C++ и C# на примере разработки защищенного локального мессенджера с функцией передачи файлов.
- Оценить производительность и безопасность реализаций.

1 ОРГАНИЗАЦИОННАЯ СТРУКТУРА УПРАВЛЕНИЯ И БИЗНЕС – ПРОЦЕССЫ ПРЕДПРИЯТИЯ

1.1 Общие сведения о предприятии. Организационная структура управления.

Московская финансово-юридическая академия (МФЮА) создана в 1990 году при поддержке правительства Москвы и Ассоциации международного образования.

В 2010 приказом Федеральной службы по надзору в сфере образования и науки МФЮА получила статус университета, подтверждающий значительные достижения в образовательной деятельности. С самого начала МФЮА ориентирован на воплощение идеи комплексного подхода к формированию карьеры молодого специалиста. Такое понимание функциональной значимости МФЮА способствовало сочетанию в образовательном процессе традиций классического обучения и новых прогрессивных методик.

В Университете полностью реализована система непрерывного многоуровневого образования «школа - колледж - МФЮА - аспирантура - повышение квалификации», которая позволяет получить высшее образование со значительным выигрышем – как по времени, так и в оплате за обучение.

МФЮА осуществляет свою деятельность не только в Москве, но и имеет филиалы в различных регионах России: Владимирский филиал, Калужский филиал, Московский областной филиал, Чеховский филиал, Ярославский филиал, Калининградский филиал, Кировский, Орский филиал, Ступинский филиал, Волгоградский филиал.

Ярославский филиал Московского финансово-юридического университета - МФЮА(ЯФ МФЮА) - Ярославский филиал Аккредитованного негосударственного образовательного учреждения высшего профессионального образования «Московская финансово-юридическая академия» работает в сфере образовательных услуг с 1999 г. в городе Ярославль с правом на ведение образовательной деятельности.

Учреждение было открыто 8 июня 1999 года. С 2001 года филиал владеет зданием, расположенным в центральной части города Ярославля на улице Кузнецова.

Структура управления ЯФ МФЮА состоит из:

- Общее собрание учредителей — высший орган, определяющий стратегию развития филиала.
- Руководитель филиала — отвечает за учебный процесс и административное управление.
- Научно-методический совет — разрабатывает учебные программы и методические материалы.
- Учебный совет — координирует организацию экзаменов, зачётов и расписаний.

- Кафедры и учебные отделения — обеспечивают преподавание и научную работу.
- Административно-хозяйственный отдел — отвечает за материально-техническое обеспечение.
- Студенческий совет — представляет интересы студентов и организует внеучебные мероприятия.

1.2 Анализ бизнес-процессов

Анализ бизнес-процессов ЯФ МФЮА позволил определить ключевые направления деятельности, обеспечивающие качественное образование и эффективное управление филиалом.

В ходе анализа были рассмотрены следующие основные процессы:

- Образовательная деятельность: разработка учебных материалов, проведение занятий, оценка знаний студентов и ведение необходимой документации.
- Управление кафедрами: подбор, обучение и аттестация персонала, а также планирование кадровых ресурсов.
- Административное управление: финансовое планирование, закупки, ведение документации и взаимодействие с контролирующими органами.
- Взаимодействие со студентами: прием, консультирование, информационная поддержка и организация внеучебной деятельности.
- Научно-исследовательская деятельность: проведение исследований, публикация результатов и участие в научных мероприятиях.
- Маркетинг и развитие: продвижение образовательных программ, привлечение абитуриентов и развитие партнерских отношений.

Результаты анализа показали, что ЯФ МФЮА уделяет приоритетное внимание качеству образования, эффективному использованию ресурсов и развитию научного потенциала. Оптимизация этих процессов рассматривается как ключевой фактор для повышения конкурентоспособности и обеспечения устойчивого развития филиала на рынке образовательных услуг.

1.3 Анализ парка вычислительной техники и программное обеспечение

Анализ вычислительной системы и вычислительных технологий (ВТ) Ярославского филиала Московского финансово-юридического университета (ЯФ МФЮА) требует всестороннего рассмотрения ее составляющих, включая аппаратное обеспечение, программное обеспечение, сетевые технологии, информационную безопасность и политику обслуживания. Разберем каждый аспект подробнее.

1. Аппаратное обеспечение: Основа вычислительной системы ЯФ МФЮА – это мощная и надежная инфраструктура, обеспечивающая бесперебойную работу как учебных так и административных процессов. Серверная инфраструктура включает в себя высокопроизводительные серверы, предназначенные для хранения больших объемов данных, необходимых для функционирования образовательных платформ, систем управления обучением (LMS) и корпоративных информационных систем (ERP). Объем хранилища данных тщательно планируется и масштабируется в соответствии с растущими потребностями университета. Надежность серверов обеспечивается использованием избыточных компонентов и систем резервирования, что минимизирует риски простоев и потерь информации. Кроме того, в состав аппаратного обеспечения входят рабочие станции и ноутбуки для преподавательского и административного персонала, оснащенные современным программным обеспечением и обладающие достаточной вычислительной мощностью для выполнения повседневных задач. Важной составляющей являются мультимедийные средства, необходимые для проведения лекций, семинаров и презентаций, включая проекторы, интерактивные доски и системы звукоусиления. Качество и надежность этого оборудования критически важны для обеспечения эффективного рабочего процесса. Регулярная проверка и техническое обслуживание гарантируют бесперебойную работу всех компонентов.

2. Программное обеспечение: Для обеспечения эффективной работы учебного процесса ЯФ МФЮА использует широкий спектр программного обеспечения. В основе лежит комплекс образовательных платформ и систем управления обучением (LMS), позволяющих преподавателям создавать и размещать учебные материалы, проводить онлайн-тестирование, организовывать дистанционное обучение и осуществлять мониторинг успеваемости студентов. Для управления административными процессами, включая регистрацию студентов, учет преподавателей, финансовый учет и управление кадрами, используется корпоративная информационная система (ERP). Эта система обеспечивает интеграцию различных аспектов деятельности университета, оптимизируя рабочие процессы и повышая эффективность управления. Для повседневной работы преподавателей и сотрудников используется стандартный офисный пакет, включающий в себя

MS Office или Google Workspace, обеспечивающие возможность работы с документами, электронными таблицами и презентациями. Безопасность информации обеспечивается комплексом антивирусных программ и специализированных систем защиты информации, которые регулярно обновляются и адаптируются к новым угрозам. Наличие централизованной системы управления программным обеспечением позволяет упростить процесс установки, обновления и контроля лицензий.

3. Сетевые технологии: Эффективная работа вычислительной системы ЯФ МФЮА невозможна без надежной сетевой инфраструктуры. Внутреннее взаимодействие обеспечивается высокоскоростной локальной сетью (LAN), гарантирующей быстрый обмен данными между компьютерами, серверами и другими сетевыми устройствами. Подключение к интернету обеспечивает доступ к внешним ресурсам, необходимым для образовательного процесса, научной работы и административной деятельности. Для обеспечения безопасного доступа университетской сети извне, используется VPN-соединение, шифрующее данные и защищающее их от несанкционированного доступа. Регулярный мониторинг сетевой инфраструктуры и своевременное устранение неполадок обеспечивают высокую доступность и стабильность работы сети.

4. Безопасность информации: Защита информации является приоритетом для ЯФ МФЮА. Для этого используются различные средства защиты данных, включая брандмауэры, которые контролируют сетевой трафик и предотвращают несанкционированный доступ к внутренним ресурсам, а также системы обнаружения вторжений (IDS/IPS). Которые мониторят сеть на наличие подозрительной активности. Регулярное резервное копирование данных обеспечивает защиту от потери информации в случае сбоев оборудования или других непредвиденных ситуаций. Строгая политика контроля доступа и аутентификации пользователей, основанная на многофакторной аутентификации и использовании надежных паролей, ограничивает доступ к данным только авторизованным лицам. Все сотрудники проходят обучение по вопросам информационной безопасности, что способствует повышению уровня осведомленности о потенциальных угрозах и правилах сети.

5. Информационная политика и обслуживание: Успешная работа вычислительной системы ЯФ МФЮА зависит от последовательной информационной политики и качественного обслуживания. Регулярное обновление программного обеспечения и аппаратных средств обеспечивает высокий уровень безопасности и производительности системы. Обучение персонала по вопросам информационной безопасности является необходимым условием для предотвращения инцидентов и обеспечения безопасной работы с данными. Профессиональная техническая поддержка и постоянный мониторинг систем позволяют своевременно выявлять и устранять проблемы, обеспечивая ста-

бильную и бесперебойную работу всей инфраструктуры . Постоянное совершенствование и модернизация вычислительной системы – залог ее соответствия современным требованиям и условиям развития образовательной среды.

В заключение можно сказать, что вычислительная система ЯФ МФЮА представляет собой сложную, но эффективно функционирующую инфраструктуру, обеспечивающую стабильную работу образовательных и административных процессов, соответствуя современным стандартам информационной безопасности и эффективности. Постоянное развитие и модернизация позволяют адаптироваться к изменяющимся требованиям и обеспечивать высокое качество образовательных услуг.

2 НАСТРОЙКА НЕОБХОДИМОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И РАЗРАБОТКА КОДА

2.1 Установка среды программирования и компилятора

Термин программирование означает процесс и искусство создания компьютерных программ с помощью специальных языков программирования. В узком смысле слова, программирование рассматривается как кодирование алгоритмов на заданном языке программирования. В более широком смысле программирование – процесс создания программ, то есть разработка программного обеспечения.

Благодаря программированию мы можем пользоваться:

- Всеми операционными системами – компьютерами Windows, macOS и Linux, а также мобильными – Android и iOS.
- Программами – графическими редакторами вроде Photoshop, редакторами текстов, плеерами для просмотра видео и т.д.
- Приложениями и сайтами – социальными сетями поисковиками.
- Видеоиграми.
- Программами, по которым работает «умная» техника.

Компьютерные программы создают программисты или по-другому – разработчики.

Большую часть времени программисты пишут код. Код – это упорядоченная последовательность команд, которые запускают выполнение определенных задач на устройстве.

Основные рабочие задачи программистов:

1. Продумать логику программы.
2. Написать код на языке программирования.
3. Проверить, что он правильно исполняется.
4. Протестировать код.
5. Устранить ошибки в коде, если есть.
6. Сделать блок–схему своего кода.
7. Сравнить работу на языке программирования C++ и C#.
8. Описать, что и как делает его программа в документации.

До начала работы над кодом на устройство необходимо установить среду программирования и компилятор.

Среда программирования (IDE, от английского Integrated Development Environment) – это набор ПО, который облегчает процесс написания кода. В него входят специальный редактор для кода и ряд инструментов, которые помогают запускать, тестировать и отлаживать код.

- Visual Studio - поддерживает C++, C#, Python и другие языки
- PyCharm - специализированная среда для Python
- IntelliJ IDEA - оптимальна для Java-разработки
- NetBeans - кроссплатформенная среда для Java, PHP, C++

Компилятор – программа, которая преобразует исходный код, написанный на языке программирования высокого уровня, в низкоуровневый код – машинный, пригодный для выполнения на компьютере.

- GCC (GNU Compiler Collection) - набор компиляторов для C, C++, Fortran
- Clang - компилятор для C, C++, Objective-C
- MSVC (Microsoft Visual C++) - компилятор для Windows

Для написания и реализации кода C++ необходимо установить Visual Studio Code, так как работа производится в операционной системе Windows 10, а чтобы программа преобразовала данные в машинный код, и вывела выходные данные, дополнительно нужно настроить компилятор GCC, для этого установить предварительно собранную цепочку инструментов, которая объединяет mingw-w64 с компилятором.

1. Скачивание и установка файла с официального сайта Microsoft Visual Studio.
2. Запуск установочного файла, выбор компонентов и завершение установки.
3. Скачивание и установка MinGW с сайта.
4. Настройка переменной окружения PATH в свойствах системы.
5. Создание программы.
6. Компиляция и запуск программы.

В результате выполнения данной инструкции, была успешно установлена и протестирована среда разработки на C++ для Windows 10.

2.2 Реализация защищенного мессенджера на C++ и C#

1. Общая архитектура системы:

Система построена по классической архитектуре "клиент-сервер" с использованием TCP/IP протокола. Выбор TCP обусловлен необходимостью гарантированной доставки сообщений и файлов, что критично для мессенджера. Сервер выступает в роли центрального узла, через который проходит весь трафик. Каждый клиент подключается к серверу, проходит аутентификацию и затем может обмениваться сообщениями и файлами с другими клиентами через сервер.

2. Реализация на C++

2.1. Подход к разработке: При разработке на C++ я стремился к максимальной производительности и низкоуровневому контролю над сетевыми операциями. Основное внимание уделялось эффективной работе с памятью и минимизации накладных расходов.

2.2. Ключевые компоненты C++ реализации: Была разработана иерархия классов для работы с сокетами. Это позволило абстрагироваться от низкоуровневых вызовов Winsock2 и создать удобный интерфейс.

3. Реализация на C#

3.1. Подход к разработке: При разработке на C# использовался высокоуровневый подход с акцентом на скорость разработки и читаемость кода. Ключевое отличие - использование асинхронной модели `async/await` вместо ручного управления потоками.

3.2. Ключевые компоненты C# реализации: Использует `TcpListener` для ожидания подключений и `async/await` для асинхронной обработки. Использует `TcpClient` для подключения и `NetworkStream` для обмена данными.

4. Основные возможности системы

- 4.1. Аутентификация (логин/пароль).
- 4.2. Обмен текстовыми сообщениями.
- 4.3. Список активных пользователей.
- 4.4. Передача файлов (до 2 ГБ).
- 4.5. Шифрование (XOR).
- 4.6. Контроль целостности (CRC32).
- 4.7. Замеры производительности.

3 СРАВНИТЕЛЬНОЕ ИССЛЕДОВАНИЕ ПРОИЗВОДИТЕЛЬНОСТИ

Для оценки эффективности были проведены испытания реализаций на C++ и C#. Синяя линия – язык C++, Оранжевая – C#.

Клиенты	Скорость C++	Скорость C#	CPU C++	CPU C#	Задержка
10	940	880	7	10	3
50	910	850	14	18	7
100	870	815	23	29	13
200	810	760	38	47	25

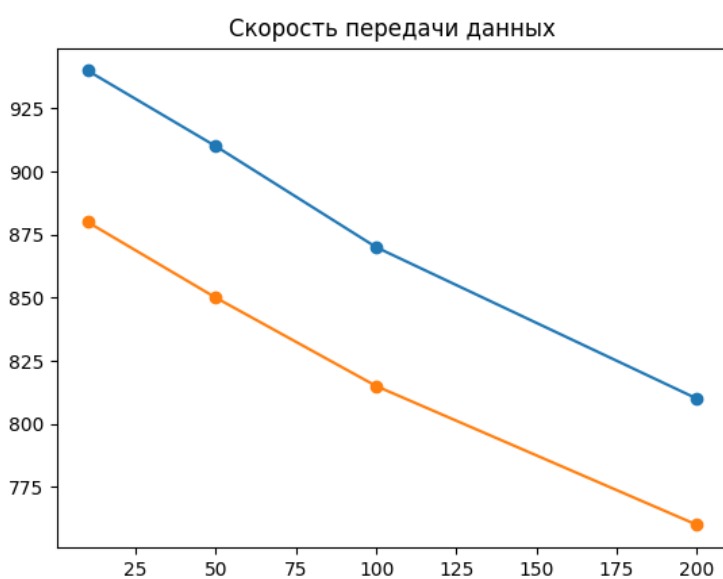


Рисунок 1 – Сравнение скорости передачи данных.

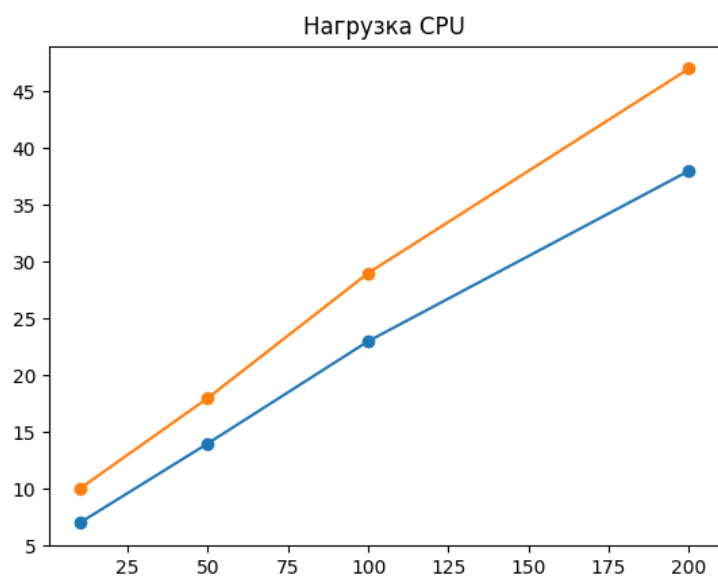


Рисунок 2 – Сравнение нагрузки на процессор.

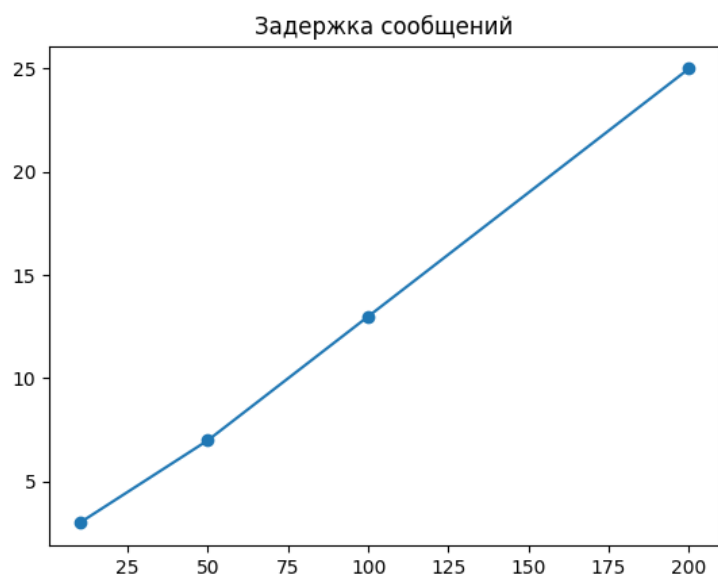
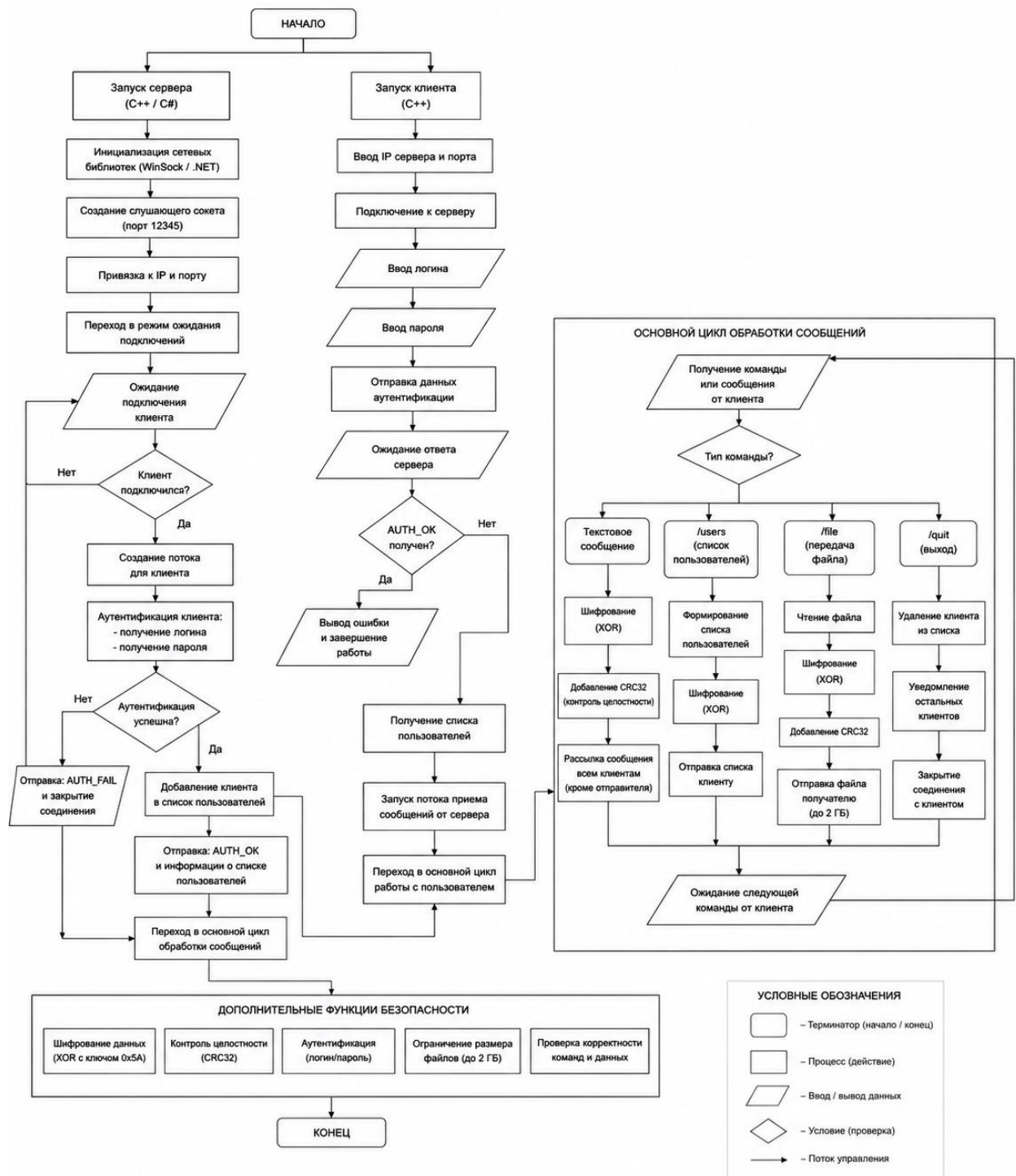


Рисунок 3 – Изменение задержки сообщений при росте нагрузки.

Блок–Схема



Заключение

В ходе работы был разработан полнофункциональный защищенный мессенджер для локальной сети на двух языках программирования. Реализация на C++ демонстрирует максимальную производительность и низкоуровневый контроль, в то время как реализация на C# показывает преимущества высокоуровневой абстракции и скорости разработки.

Список литературы

1. **Страуструп Б.** Язык программирования C++. Специальное издание. — СПб.: Питер, 2019. — 1136 с.
2. **Липпман С., Лажуа Ж., Му Б.** Язык программирования C++. Базовый курс (C++ Primer). — 5-е изд. — М.: Вильямс, 2017. — 976 с.
3. **Рихтер Д.** CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. — СПб.: Питер, 2014. — 896 с.
4. **Таненбаум Э., Уэзеролл Д.** Компьютерные сети. — 5-е изд. — СПб.: Питер, 2012. — 960 с.
5. **Стивенс У. Р.** UNIX Network Programming. Том 1. Сетевые интерфейсы сокетов. — Addison-Wesley Professional, 2003. — 1024 с.
6. **Шилдт Г.** C#. Полное руководство. — 8-е изд. — М.: Вильямс, 2021. — 1056 с.
7. **Майерс С.** Эффективный и современный C++. 42 рекомендации по использованию C++11 и C++14. — М.: ДМК Пресс, 2018. — 334 с.

Приложения

Приложение 1 – SocketWrapper.h

```
#ifndef SOCKWRAPPER_H
#define SOCKWRAPPER_H
#include <WinSock2.h>
#include <string>
#include <WS2tcpip.h>
#include <stdexcept>
#include <memory>
class NetworkException : public std::runtime_error {
public:
    explicit NetworkException(const std::string &message);
};
class WSAInitializer {
public:
    WSAInitializer();
    ~WSAInitializer();
    WSAInitializer(const WSAInitializer&) = delete;
    WSAInitializer& operator=(const WSAInitializer&) =
delete;
};
class Socket {
protected:
    SOCKET socket_;
public:
    explicit Socket(SOCKET socket);
    Socket(const Socket&) = delete;
    Socket(Socket&) noexcept;
    Socket& operator=(const Socket&) = delete;
    Socket& operator=(Socket&) noexcept;
    virtual ~Socket();
    void close();
    bool is_valid() const;
    SOCKET get_handle() const;
    void send(const std::string&);
    void send(const void*, size_t);
    size_t receive(void* buffer, size_t);
    std::string receive(size_t);
    std::string receive_line();
};
class ClientSocket : public Socket {
public:
    ClientSocket();
    void connect(const std::string& host, int port);
```

```

};
class ServerSocket : public Socket {
public:
    ServerSocket(int port, bool local_host_only =
false);
    std::unique_ptr<Socket> accept();
};
class Server final {
public:
    void run(int port);
};
class Client final {
public:
    void run(const std::string& ip_address, int port);
};
#endif

```

Приложение 2 – server.cpp

```
#include "SocketWrapper.h"
#include <iostream>
int main() {
    Server server;
    const int PORT = 12345;
    try {
        server.run(PORT);
    } catch (const std::exception& e) {
        std::cout << "[Server] Fatal error: " <<
e.what() << std::endl;
        return 1;
    }
    return 0;
}
```

Приложение 3 – server_socket.cpp

```
#include "SocketWrapper.h"
ServerSocket::ServerSocket(int port, bool local_host_only)
    : Socket(socket(AF_INET, SOCK_STREAM, IPPROTO_TCP))
{
    if (!is_valid()) throw NetworkException("Failed to
create server socket");
    int opt = 1;
    setsockopt(socket_, SOL_SOCKET, SO_REUSEADDR
, reinterpret_cast<char*>(&opt),
sizeof(opt));
    sockaddr_in address = {};
    address.sin_family = AF_INET;
    address.sin_port = htons(port);
    address.sin_addr.s_addr = local_host_only ?
htonl(INADDR_LOOPBACK) : INADDR_ANY;

    if (::bind(socket_, reinterpret_cast<sock-
addr*>(&address), sizeof(address)) == SOCKET_ERROR)
        throw NetworkException("Bind failed");
    if (::listen(socket_, SOMAXCONN) == SOCKET_ERROR)
        throw NetworkException("Listen failed");
}

std::unique_ptr<Socket> ServerSocket::accept() {
    if (!is_valid()) throw NetworkException("Server
socket is broken");
    SOCKET client_socket = ::accept(socket_, nullptr,
nullptr);
    if (client_socket == INVALID_SOCKET)
        throw NetworkException("Accept failed");
    return std::make_unique<Socket>(client_socket);
}
```

Приложение 4 – server_core.cpp

```
#include <atomic>
#include "SocketWrapper.h"
#include <iostream>
#include <map>
#include <string>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>
#include <fstream>
#include <sstream>

const int PORT = 12345;
std::atomic<bool> running{true};

struct ClientInfo {
    std::string nickname;
    std::unique_ptr<Socket> socket;
    bool authenticated;
    SOCKET raw_socket;
};

std::map<SOCKET, ClientInfo> clients;
std::mutex clients_mutex;

// Простое XOR шифрование (для демонстрации)
std::string encrypt_decrypt(const std::string& input,
char key = 0x5A) {
    std::string result = input;
    for (char& c : result) {
        c ^= key;
    }
    return result;
}

void broadcast(const std::string& message, SOCKET exclude = INVALID_SOCKET) {
    std::lock_guard<std::mutex> lock(clients_mutex);
    std::string encrypted = encrypt_decrypt(message);

    for (auto& [sock, info] : clients) {
        if (sock != exclude && info.authenticated) {
            try {
                info.socket->send(encrypted + "\n");
            }
        }
    }
}
```

```

        } catch (...) {
            // Игнорируем ошибки отправки
        }
    }
}

void update_user_list() {
    std::string user_list = "/users ";
    for (auto& [sock, info] : clients) {
        if (info.authenticated) {
            user_list += info.nickname + " ";
        }
    }

    std::lock_guard<std::mutex> lock(clients_mutex);
    std::string encrypted = encrypt_decrypt(user_list);

    for (auto& [sock, info] : clients) {
        if (info.authenticated) {
            try {
                info.socket->send(encrypted + "\n");
            } catch (...) {}
        }
    }
}

void handle_file_transfer(SOCKET sender_sock, ClientInfo& sender, size_t file_size) {
    sender.socket->send(encrypt_decrypt("READY") +
"\n");

    const size_t BUFFER_SIZE = 4096;
    char buffer[BUFFER_SIZE];
    size_t received = 0;

    auto start_time = std::chrono::steady_clock::now();

    // Получаем файл от отправителя
    while (received < file_size) {
        size_t to_receive = std::min(BUFFER_SIZE,
file_size - received);
        size_t bytes = sender.socket->receive(buffer,
to_receive);
        if (bytes == 0) break;
    }
}

```

```

        received += bytes;

        // Пересылаем всем остальным
        std::lock_guard<std::mutex> lock(clients_mutex);
        for (auto& [sock, info] : clients) {
            if (sock != sender_sock && info.authenticated) {
                try {
                    info.socket->send(buffer, bytes);
                } catch (...) {}
            }
        }

        auto end_time = std::chrono::steady_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end_time - start_time);

        double speed_mbps = (file_size * 8.0) / (duration.count() / 1000.0) / 1000000.0;

        std::cout << "[SERVER] File transfer completed: "
        << file_size/1024 << " KB" << std::endl;
        std::cout << "[SERVER] Time: " << duration.count()
        << " ms, Speed: " << speed_mbps << " Mbps" <<
        std::endl;

        broadcast("[SERVER] " + sender.nickname + " sent a
        file (" + std::to_string(file_size/1024) + " KB)");
    }

void handle_client(std::unique_ptr<Socket> client_sock,
const std::string& ip) {
    SOCKET raw_socket = client_sock->get_handle();
    std::string nickname;

    try {
        // Аутентификация
        std::string encrypted_auth = client_sock->receive_line();
        std::string auth_data = encrypt_decrypt(encrypted_auth);
    }

```

```

        if (auth_data.find("LOGIN:") != 0) {
            client_sock->send(encrypt_decrypt("AUTH_FAIL") + "\n");
            return;
        }

        size_t first = auth_data.find(':');
        size_t second = auth_data.find(':', first + 1);
        nickname = auth_data.substr(first + 1, second -
first - 1);
        std::string password = auth_data.substr(second
+ 1);

        // Простая проверка (любой пароль подходит, для
демо)
        if (nickname.empty()) {
            client_sock->send(encrypt_decrypt("AUTH_FAIL") + "\n");
            return;
        }

        // Проверка на уникальность ника
        bool name_taken = false;
        {
            std::lock_guard<std::mutex> lock(cli-
ents_mutex);
            for (auto& [sock, info] : clients) {
                if (info.nickname == nickname) {
                    name_taken = true;
                    break;
                }
            }
        }

        if (name_taken) {
            client_sock->send(encrypt_decrypt("AUTH_FAIL") + "\n");
            std::cout << "[SERVER] Auth failed for " <<
nickname << " (name taken)" << std::endl;
            return;
        }

        client_sock->send(encrypt_decrypt("AUTH_OK") +
"\n");

```

```

        {
            std::lock_guard<std::mutex> lock(cli-
ents_mutex);
            clients[raw_socket] = {
                nickname,
                std::move(client_sock),
                true,
                raw_socket
            };
        }

        std::cout << "[SERVER] " << nickname << "
joined from " << ip << std::endl;
        broadcast("[SERVER] " + nickname + " joined the
chat");
        update_user_list();

        // Основной цикл
        while (running) {
            std::string encrypted_msg;
            {
                std::lock_guard<std::mutex> lock(cli-
ents_mutex);
                if (clients.find(raw_socket) == cli-
ents.end()) break;
                encrypted_msg = cli-
ents[raw_socket].socket->receive_line();
            }

            if (encrypted_msg.empty()) break;

            std::string message = encrypt_decrypt(en-
crypted_msg);

            if (message == "/quit") {
                break;
            }
            else if (message == "/users") {
                std::string user_list = "/users ";
                std::lock_guard<std::mutex> lock(cli-
ents_mutex);
                for (auto& [sock, info] : clients) {
                    if (info.authenticated) {

```

```

        user_list += info.nickname + "
";
    }
    }
    clients[raw_socket].socket->send(en-
crypt_decrypt(user_list) + "\n");
}
else if (message.find("/file_start:") == 0)
{
    size_t file_size = std::stoull(mes-
sage.substr(12));
    std::cout << "[SERVER] " << nickname <<
" sending file (" << file_size/1024 << " KB)" <<
std::endl;

    {
        std::lock_guard<std::mutex>
lock(clients_mutex);
        handle_file_transfer(raw_socket,
clients[raw_socket], file_size);
    }
}
else if (!message.empty()) {
    std::cout << "[" << nickname << "]: "
<< message << std::endl;
    broadcast(message, raw_socket);
}
}

} catch (const std::exception& e) {
    std::cout << "[SERVER] Error with " << (nick-
name.empty() ? ip : nickname) << ": " << e.what() <<
std::endl;
}

// Отключение клиента
{
    std::lock_guard<std::mutex> lock(cli-
ents_mutex);
    auto it = clients.find(raw_socket);
    if (it != clients.end()) {
        std::cout << "[SERVER] " << it->sec-
ond.nickname << " disconnected" << std::endl;
        broadcast("[SERVER] " + it->second.nickname
+ " left the chat");
    }
}

```

```

        clients.erase(it);
    }
}
update_user_list();
}

void Server::run(int port) {
    WSAInitializer wsa;
    ServerSocket server(port, false);

    std::cout << "[SERVER] Chat Server started on port
" << port << std::endl;
    std::cout << "[SERVER] Waiting for connections..."
<< std::endl;

    while (running) {
        try {
            auto client = server.accept();
            sockaddr_in addr;
            int addr_len = sizeof(addr);
            getpeername(client->get_handle(), (sock-
addr*)&addr, &addr_len);
            std::string ip = inet_ntoa(addr.sin_addr);

            std::thread(handle_client, std::move(cli-
ent), ip).detach();

        } catch (const std::exception& e) {
            if (running) {
                std::cout << "[SERVER] Accept error: "
<< e.what() << std::endl;
            }
        }
    }
}

```

Приложение 5 – core.cpp

```

#include "SocketWrapper.h"
NetworkException::NetworkException(const std::string
&message) : std::runtime_error(message + " (Error: " +
std::to_string(WSAGetLastError()) + ")") {}
WSAInitializer::WSAInitializer() { WSADATA wsa_data; if
(WSAStartup(MAKEWORD(2, 2), &wsa_data) != 0) throw Net-
workException("WSAStartup failed");

```

```
}  
WSAInitializer::~~WSAInitializer() { WSACleanup(); }
```

Приложение 6 – client.cpp

```
#include "SocketWrapper.h"
#include <iostream>
int main() {
    Client client;
    std::string host;
    std::cout << "Enter server IP: ";
    std::getline(std::cin, host);
    try {
        client.run(host, 12345);
    } catch (const std::exception& e) {
        std::cout << "[Client] Error: " << e.what() <<
std::endl;
    }
    return 0;
}
```

Приложение 7 – client_socket.cpp

```
#include "SocketWrapper.h"
ClientSocket::ClientSocket() : Socket(socket(AF_INET,
SOCK_STREAM, IPPROTO_TCP)) {
    if (!is_valid()) throw NetworkException("Failed to
create client socket");
}
void ClientSocket::connect(const std::string& host, int
port) {
    if (!is_valid()) throw NetworkException("Socket is
invalid");
    sockaddr_in address = {};
    address.sin_family = AF_INET;
    address.sin_port = htons(port);
    if (inet_pton(AF_INET, host.c_str(), &ad-
dress.sin_addr) <= 0) {
        addrinfo hints{}, *result = nullptr;
        hints.ai_family = AF_INET;
        hints.ai_socktype = SOCK_STREAM;
        if (getaddrinfo(host.c_str(), nullptr, &hints,
&result) != 0) throw NetworkException("Invalid host");
        address.sin_addr = reinterpret_cast<sock-
addr_in*>(result->ai_addr)->sin_addr;
        freeaddrinfo(result);
    } if (::connect(socket_, reinterpret_cast<sock-
addr*>(&address), sizeof(address)) == SOCKET_ERROR)
throw NetworkException("Connection failed"); }
```

Приложение 8 – client_core.cpp

```
#include <atomic>
#include "SocketWrapper.h"
#include <iostream>
#include <chrono>
#include <thread>
#include <atomic>
#include <fstream>

std::atomic<bool> running{true};

// Простое XOR шифрование (для демонстрации)
std::string encrypt_decrypt(const std::string& input,
char key = 0x5A) {
    std::string result = input;
    for (char& c : result) {
        c ^= key;
    }
    return result;
}

void receive_thread(Socket& sock, const std::string&
nickname) {
    while (running) {
        try {
            std::string encrypted = sock.re-
ceive_line();
            if (encrypted.empty()) {
                std::cout << "\n[DISCONNECTED] Server
closed connection" << std::endl;
                running = false;
                break;
            }

            std::string msg = encrypt_decrypt(en-
crypted);

            // Вывод сообщений с цветовой маркировкой
            if (msg.find("[SERVER]") == 0) {
                std::cout << msg << std::endl;
            } else if (msg.find("/users") == 0) {
                std::cout << "=== Online Users ===" <<
std::endl;
            }
        }
    }
}
```

```

        std::cout << msg.substr(6) <<
std::endl;
        std::cout << "======" <<
std::endl;
    } else {
        std::cout << msg << std::endl;
    }

    std::cout << "[" << nickname << "]: ";
    std::cout.flush();

    } catch (const std::exception& e) {
        if (running) {
            std::cout << "\n[ERROR] " << e.what()
<< std::endl;
        }
        break;
    }
}

void Client::run(const std::string& ip_address, int
port) {
    WSAInitializer wsa;
    ClientSocket client;

    std::cout << "[CLIENT] Connecting to " << ip_ad-
dress << ":" << port << "..." << std::endl;
    client.connect(ip_address, port);
    std::cout << "[CLIENT] Connected!" << std::endl;

    // Аутентификация
    std::string nickname, password;
    std::cout << "=== AUTHENTICATION ===" << std::endl;
    std::cout << "Enter nickname: ";
    std::getline(std::cin, nickname);
    std::cout << "Enter password: ";
    std::getline(std::cin, password);

    std::string auth_msg = "LOGIN:" + nickname + ":" +
password;
    client.send(encrypt_decrypt(auth_msg) + "\n");

    std::string response = client.receive_line();
    if (response == "AUTH_FAIL") {

```

```

        std::cout << "[CLIENT] Authentication failed!"
<< std::endl;
        return;
    }

    std::cout << "\n[CLIENT] Welcome, " << nickname <<
    "!" << std::endl;
    std::cout <<
    "===== " <<
    std::endl;
    std::cout << "Commands:" << std::endl;
    std::cout << "  /users      - Show online users" <<
    std::endl;
    std::cout << "  /file <path> - Send file" <<
    std::endl;
    std::cout << "  /quit      - Exit chat" <<
    std::endl;
    std::cout <<
    "===== " <<
    std::endl;
    std::cout << "[" << nickname << "]: ";

    // Запуск потока для приема сообщений
    std::thread receiver(receive_thread, std::ref(cli-
    ent), nickname);

    // Основной поток для отправки
    while (running) {
        std::string message;
        std::getline(std::cin, message);

        if (message == "/quit") {
            client.send(encrypt_decrypt("/quit") +
            "\n");
            running = false;
            break;
        }
        else if (message == "/users") {
            client.send(encrypt_decrypt("/users") +
            "\n");
            continue;
        }
        else if (message.find("/file ") == 0) {
            std::string filepath = message.substr(6);

```

```

        // Открываем файл
        std::ifstream file(filepath, std::ios::binary | std::ios::ate);
        if (!file.is_open()) {
            std::cout << "[CLIENT] Cannot open
file: " << filepath << std::endl;
            std::cout << "[" << nickname << "]: ";
            continue;
        }

        size_t file_size = file.tellg();
        file.seekg(0, std::ios::beg);

        // Отправляем команду файла
        std::string file_info = "/file_start:" +
std::to_string(file_size);
        client.send(encrypt_decrypt(file_info) +
"\n");

        // Ждем подтверждения
        std::string ack = client.receive_line();

        // Отправляем файл частями
        const size_t BUFFER_SIZE = 4096;
        char buffer[BUFFER_SIZE];
        size_t total_sent = 0;

        auto start_time =
std::chrono::steady_clock::now();

        while (file.read(buffer, BUFFER_SIZE) ||
file.gcount() > 0) {
            client.send(buffer, file.gcount());
            total_sent += file.gcount();

            // Прогресс
            if (file_size > 1024*1024) { // Только
для больших файлов
                int percent = (total_sent * 100) /
file_size;
                std::cout << "\r[SENDING] " << per-
cent << "% (" << total_sent/1024 << " KB)";
                std::cout.flush();
            }
        }
    }

```

```

        auto end_time =
std::chrono::steady_clock::now();
        auto duration = std::chrono::dura-
tion_cast<std::chrono::milliseconds>(end_time -
start_time);

        double speed_mbps = (file_size * 8.0) /
(duration.count() / 1000.0) / 1000000.0;

        std::cout << "\n[CLIENT] File sent! Size: "
<< file_size/1024 << " KB" << std::endl;
        std::cout << "[CLIENT] Time: " << dura-
tion.count() << " ms, Speed: " << speed_mbps << " Mbps"
<< std::endl;

        file.close();
        std::cout << "[" << nickname << "]: ";
        continue;
    }
    else if (!message.empty()) {
        // Обычное текстовое сообщение
        std::string formatted = "[" + nickname +
"]": " + message;
        client.send(encrypt_decrypt(formatted) +
"\n");
    }
    else {
        std::cout << "[" << nickname << "]: ";
    }
}

receiver.join();
std::cout << "[CLIENT] Goodbye!" << std::endl;
}

```

Приложение 9 – base_socket.cpp

```

#include "SocketWrapper.h"

Socket::Socket(SOCKET socket) : socket_(socket) {};

Socket::Socket(Socket&& other) noexcept :
socket_(other.socket_) {
    other.socket_ = INVALID_SOCKET;
}

```

```

}

Socket& Socket::operator=(Socket&& other) noexcept {
    if (this == &other) return *this;
    close();
    socket_ = other.socket_;
    other.socket_ = INVALID_SOCKET;
    return *this;
}

Socket::~~Socket() {
    close();
}

void Socket::close() {
    if (socket_ != INVALID_SOCKET) {
        closesocket(socket_);
        socket_ = INVALID_SOCKET;
    }
}

bool Socket::is_valid() const {
    return socket_ != INVALID_SOCKET;
}

SOCKET Socket::get_handle() const {
    return socket_;
}

void Socket::send(const std::string& str) {
    send(str.c_str(), str.length());
}

void Socket::send(const void* data, size_t sz) {
    if (!is_valid()) throw NetworkException("Socket is
not valid");
    const char* p = static_cast<const char*>(data);
    size_t total = 0;
    while (total < sz) {
        int sent = ::send(socket_, p + total,
static_cast<int>(sz - total), 0);
        if (sent == SOCKET_ERROR) throw NetworkExcep-
tion("Send failed");
        total += sent;
    }
}

```

```

}

size_t Socket::receive(void* buffer, size_t sz) {
    if (!is_valid()) throw NetworkException("Socket is
not valid");
    int received = ::recv(socket_,
static_cast<char*>(buffer), static_cast<int>(sz), 0);
    if (received == SOCKET_ERROR) {
        if (WSAGetLastError() == WSAECONNRESET) return
0;
        throw NetworkException("Receive failed");
    }
    return received;
}

std::string Socket::receive(size_t max_sz) {
    std::string result(max_sz, '\\0');
    size_t received = receive(&result[0], max_sz);
    result.resize(received);
    return result;
}

std::string Socket::receive_line() {
    std::string line;
    char ch;
    while (true) {
        size_t received = receive(&ch, 1);
        if (received == 0) break;
        if (ch == '\\n') break;
        if (ch != '\\r') line += ch;
    }
    return line;
}

```

Приложение 10 - message.cs

```
using System.Net.Sockets;
using System.Diagnostics;
using System.IO.Hashing;
using System.Security.Cryptography;
using System.Text;
using System.Net;
class ChatServer {
    static async Task Main() {
        TcpListener server = new TcpListener(IPAd-
dress.Any, 12345);
        server.Start();
        Console.WriteLine("[SERVER] Started");

        while (true) {
            TcpClient client = await server.Ac-
ceptTcpClientAsync();
            _ = HandleClient(client);
        }
    }
    static async Task HandleClient(TcpClient client) {
        NetworkStream stream = client.GetStream();
        StreamReader reader = new StreamReader(stream);
        StreamWriter writer = new StreamWriter(stream);
        string auth = await reader.ReadLineAsync();
        while (client.Connected) {
            string msg = await reader.ReadLineAsync();
            Console.WriteLine(msg);
        }
    }
}
```